

口罩佩戴检测

1、实验介绍

1.1 实验背景

今年一场席卷全球的新型冠状病毒给人们带来了沉重的生命财产的损失。

有效防御这种传染病的方法就是积极佩戴口罩。

我国对此也采取了严肃的措施，在公共场合要求人们必须佩戴口罩。

在本次实验中，我们要建立一个目标检测的模型，可以识别图中的人是否佩戴了口罩。

1.2 实验要求

- 1) 建立深度学习模型，检测出图中的人是否佩戴了口罩，并将其尽可能调整到最佳状态。
- 2) 学习经典的模型 MTCNN 和 MobileNet 的结构。
- 3) 学习训练时的方法。

1.3 实验环境

可以使用基于 Python 的 OpenCV、PIL 库进行图像相关处理，使用 Numpy 库进行相关数值运算，使用 Keras 等深度学习框架训练模型等。

1.4 实验思路

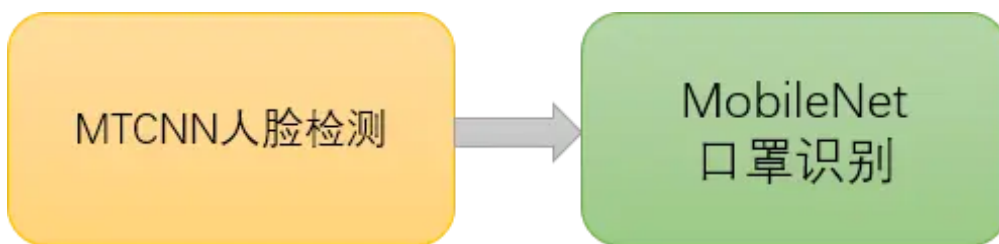
针对目标检测的任务，可以分为两个部分：目标识别和位置检测。

通常情况下，特征提取需要由特有的特征提取神经网络来完成，如 VGG、MobileNet、ResNet 等，这些特征提取网络往往被称为 Backbone。而在 Backbone 后面接全连接层(FC)就可以执行分类任务。

但 FC 对目标的位置识别乏力。经过算法的发展，当前主要以特定的功能网络来代替 FC 的作用，如 Mask-Rcnn、SSD、YOLO 等。

我们选择充分使用已有的人脸检测的模型，再训练一个识别口罩的模型，从而提高训练的开支、增强模型的准确率。

本次案例：



2、数据集

2.1、数据集介绍

数据信息存放在 `/datasets/5f680a696ec9b83bb0037081-momodel/data` 文件夹下。

该文件夹主要有文件夹 `image`、文件 `train.txt`、文件夹 `keras_model_data` 和文件夹 `mindspore_model_data` 共四部分：

- **image 文件夹**：图片分成两类，戴口罩的和没有戴口罩的
- **train.txt**：存放的是 image 文件夹下对应图片的标签（keras 框架专用文件）
- **keras_model_data 文件夹**：存放 keras 框架相关预训练好的模型（keras 框架专用文件夹）

- **mindspore_model_data** 文件夹：存放 mindspore 框架相关预训练好的模型（mindspore 框架专用文件夹）

数据集包含正负样本，（正样本佩戴口罩，负样本未戴口罩）

在使用前我们需要对数据集进行处理，包括图片的尺寸变换，数据增强等。

2.2、制作训练时所需的批量数据集

图片生成器 [ImageDataGenerator](#): keras.preprocessing.image 模块中的图片生成器，主要用以生成一个 batch 的图像数据，支持实时数据提升。训练时该函数会无限生成数据，直到达到规定的 epoch 次数为止。同时也可以对数据进行增强，扩充数据集大小，增强模型的泛化能力，比如进行旋转，变形，归一化等等。

图片生成器的主要方法：

- `fit(x, augment=False, rounds=1)`: 计算依赖于数据的变换所需要的统计信息(均值方差等)。
- `flow(self, X, y, batch_size=32, shuffle=True, seed=None, save_to_dir=None, save_prefix="", save_format='png')`: 接收 Numpy 数组和标签为参数,生成经过数据提升或标准化后的 batch 数据，并在一个无限循环中不断的返回 batch 数据。
- `flow_from_directory(directory)`: 以文件夹路径为参数，会从路径推测 label，生成经过数据提升/归一化后的数据，在一个无限循环中无限产生 batch 数据。

英文参考链接：<https://keras.io/preprocessing/image/>

中文参考链接：<https://keras-cn.readthedocs.io/en/latest/preprocessing/image/>

以上只是对图片生成器进行简单的介绍，详细信息请参考中英文链接。

根据上面的介绍和我们数据集的特性，我们主要运用 `ImageDataGenerator()` 和

`flow_from_directory()` 方法。我们将数据处理过程封装成为一个函数：

3、MTCNN：人脸检测

3.1 MTCNN 解读

参考文献：《Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks》

文献与代码地址：https://kpzhang93.github.io/MTCNN_face_detection_alignment/

论文的主要贡献：

- 1) **三阶段的级联 (cascaded) 架构**
- 2) **coarse-to-fine 的方式**
- 3) **new online hard sample mining 策略**
- 4) **同时进行人脸检测和人脸对齐**
- 5) **state-of-the-art 性能**

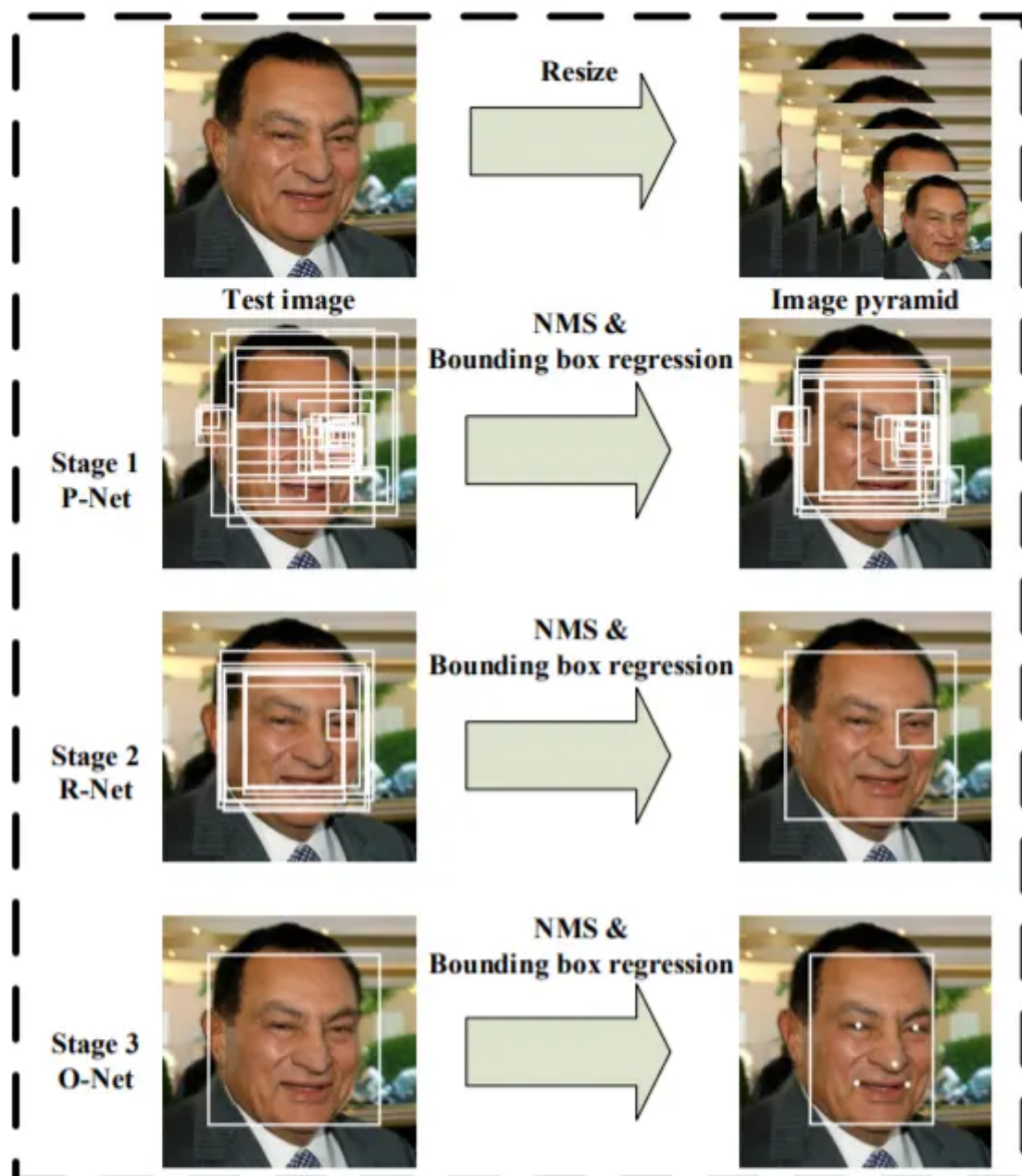


Fig. 1. Pipeline of our cascaded framework that includes three-stage multi-task deep convolutional networks. Firstly, candidate windows are produced through a fast Proposal Network (P-Net). After that, we refine these candidates in the next stage through a Refinement Network (R-Net). In the third stage, The Output Network (O-Net) produces final bounding box and facial landmarks position.

3.2 MTCNN 的使用

直接使用现有的表现较好的 MTCNN 的三个权重文件，它们已经保存在

`datasets/5f680a696ec9b83bb0037081-momodel/data/keras_model_data` 文件夹下，路径如下：

```
pnet_path = "./datasets/5f680a696ec9b83bb0037081-  
momodel/data/keras_model_data/pnet.h5"  
rnet_path = "./datasets/5f680a696ec9b83bb0037081-  
momodel/data/keras_model_data/rnet.h5"  
onet_path = "./datasets/5f680a696ec9b83bb0037081-  
momodel/data/keras_model_data/onet.h5"
```

通过搭建 MTCNN 网络实现人脸检测（搭建模型py文件在 `keras_py` 文件夹）

- keras_py/mtcnn.py 文件是在搭建 MTCNN 网络。
- keras_py/face_rec.py 文件是在绘制人脸检测的矩形框。

4、口罩识别

```
# 加载 MobileNet 的预训练模型权重
weights_path = basic_path + 'keras_model_data/mobilenet_1_0_224_tf_no_top.h5'
# 图像数据的行数和列数
height, width = 160, 160
model = MobileNet(input_shape=[height,width,3],classes=2)
model.load_weights(weights_path,by_name=True)
print('加载完成...')
```

4.1 手动调整学习率(可调参)

学习率的手动设置可以使模型训练更加高效。

这里我们设置当模型在两轮迭代后，准确率没有上升，就调整学习率。

```
# 学习率下降的方式，acc三次不下降就下降学习率继续训练
reduce_lr = ReduceLRonPlateau(
    monitor='acc', # 检测的指标
    factor=0.5, # 当acc不下降时将学习率下调的比例
    patience=2, # 检测轮数是每隔两轮
    verbose=2 # 信息展示模式
)
```

4.2 早停法(可调参)

当我们训练深度学习神经网络的时候通常希望能获得最好的泛化性能。

但是所有的标准深度学习神经网络结构如全连接多层感知机都很容易过拟合。

当网络在训练集上表现越来越好，错误率越来越低的时候，就极有可能出现了过拟合。

早停法就是当我们在检测到这一趋势后，就停止训练，这样能避免继续训练导致过拟合的问题。

```
early_stopping = EarlyStopping(
    monitor='val_loss', # 检测的指标
    min_delta=0, # 增大或减小的阈值
    patience=10, # 检测的轮数频率
    verbose=1 # 信息展示的模式
)
```

4.3 训练模型

```
# 一次的训练集大小
batch_size = 8
# 图片数据路径
data_path = basic_path + 'image'
# 图片处理
train_generator,test_generator = processing_data(data_path, height=160,
width=160, batch_size=batch_size, test_split=0.1)
# 编译模型
model.compile(loss='binary_crossentropy', # 二分类损失函数
    optimizer=Adam(lr=0.1), # 优化器
    metrics=['accuracy']) # 优化目标
# 训练模型
```

```

history = model.fit(train_generator,
                    epochs=3, # epochs: 整数，数据的迭代总轮数。
                    # 一个epoch包含的步数,通常应该等于你的数据集的样本数量除以批量大小。
                    steps_per_epoch=637 // batch_size,
                    validation_data=test_generator,
                    validation_steps=70 // batch_size,
                    initial_epoch=0, # 整数。开始训练的轮次（有助于恢复之前的训练）。
                    callbacks=[checkpoint_period, reduce_lr])

# 保存模型
model.save_weights(model_dir + 'temp.h5')

```

5、实验结果

模型的训练和预测

首先将例程给的代码进行运行，发现训练效果一般，于是提高为学习率和训练轮数，令 $lr = 0.0001$ 、 $epochs = 30$ ，得到的情况如下所示

测试详情

测试点	状态	时长	结果
在 5 张图片上 测试模型	✓	18s	得分:90.83

确定

发现是因为识别人数错误，通过修改 # 门限函数 `self.threshold = [0.5,0.6,0.8]` 里面的数值，可以对框选的条件进行修改，经过调参，最终确定修改其为 `[0.4,0.6,0.65]`。结果如下图所示，效果很好。

测试详情

测试点	状态	时长	结果
在 5 张图片上 测试模型	✓	20s	得分:97.5

确定

6、附录

train.py

```

import warnings
# 忽视警告
warnings.filterwarnings('ignore')
import os
import matplotlib

```

```

import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau,
EarlyStopping
from tensorflow.keras.applications.imagenet_utils import preprocess_input
from tensorflow.keras import backend as K
from tensorflow.keras.optimizers import Adam
from keras.utils import np_utils, get_file

K.image_data_format() == 'channels_last'

from keras_py.utils import get_random_data
from keras_py.face_rec import mask_rec
from keras_py.face_rec import face_rec
from keras_py.mobileNet import MobileNet

# 数据集路径
basic_path = "./datasets/5f680a696ec9b83bb0037081-momodel/data/"

# 导入图片生成器
from tensorflow.keras.preprocessing.image import ImageDataGenerator

def processing_data(data_path, height, width, batch_size=32, test_split=0.1):
    """
    数据处理
    :param data_path: 带有子目录的数据集路径
    :param height: 图像形状的行数
    :param width: 图像形状的列数
    :param batch_size: batch 数据的大小，整数，默认32。
    :param test_split: 在 0 和 1 之间浮动。用作测试集的训练数据的比例，默认0.1。
    :return: train_generator, test_generator: 处理后的训练集数据、验证集数据
    """

    train_data = ImageDataGenerator(
        # 对图片的每个像素值均乘上这个放缩因子，把像素值放缩到0和1之间有利于模型的收敛
        rescale=1. / 255,
        # 浮点数，剪切强度（逆时针方向的剪切变换角度）
        shear_range=0.1,
        # 随机缩放的幅度，若为浮点数，则相当于[lower,upper] = [1 - zoom_range,
1+zoom_range]
        zoom_range=0.1,
        # 浮点数，图片宽度的某个比例，数据提升时图片水平偏移的幅度
        width_shift_range=0.1,
        # 浮点数，图片高度的某个比例，数据提升时图片竖直偏移的幅度
        height_shift_range=0.1,
        # 布尔值，进行随机水平翻转
        horizontal_flip=True,
        # 布尔值，进行随机竖直翻转
        vertical_flip=True,
        # 在 0 和 1 之间浮动。用作验证集的训练数据的比例
        validation_split=test_split
    )

    # 接下来生成测试集，可以参考训练集的写法
    test_data = ImageDataGenerator(
        rescale=1. / 255,

```



```

        validation_split=test_split)

train_generator = train_data.flow_from_directory(
    # 提供的路径下面需要有子目录
    data_path,
    # 整数元组 (height, width), 默认: (256, 256)。 所有的图像将被调整到的尺寸。
    target_size=(height, width),
    # 一批数据的大小
    batch_size=batch_size,
    # "categorical", "binary", "sparse", "input" 或 None 之一。
    # 默认: "categorical",返回one-hot 编码标签。
    class_mode='categorical',
    # 数据子集 ("training" 或 "validation")
    subset='training',
    seed=0)

test_generator = test_data.flow_from_directory(
    data_path,
    target_size=(height, width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=0)

    return train_generator, test_generator

# 数据路径
data_path = basic_path + 'image'

# 图像数据的行数和列数
height, width = 160, 160

# 获取训练数据和验证数据集
train_generator, test_generator = processing_data(data_path, height, width)

pnet_path = "./datasets/5f680a696ec9b83bb0037081-
momodel/data/keras_model_data/pnet.h5"
rnet_path = "./datasets/5f680a696ec9b83bb0037081-
momodel/data/keras_model_data/rnet.h5"
onet_path = "./datasets/5f680a696ec9b83bb0037081-
momodel/data/keras_model_data/onet.h5"

# 加载 MobileNet 的预训练模型权重
weights_path = basic_path + 'keras_model_data/mobilenet_1_0_224_tf_no_top.h5'
# 图像数据的行数和列数
height, width = 160, 160
model = MobileNet(input_shape=[height,width,3],classes=2)
model.load_weights(weights_path,by_name=True)
print('加载完成...')

def save_model(model, checkpoint_save_path, model_dir):
    """
    保存模型，每迭代3次保存一次
    :param model: 训练的模型
    :param checkpoint_save_path: 加载历史模型
    :param model_dir:
    :return:
    """

```

```

if os.path.exists(checkpoint_save_path):
    print("模型加载中")
    model.load_weights(checkpoint_save_path)
    print("模型加载完毕")
checkpoint_period = ModelCheckpoint(
    # 模型存储路径
    model_dir + 'ep{epoch:03d}-loss{loss:.3f}-val_loss{val_loss:.3f}.h5',
    # 检测的指标
    monitor='val_acc',
    # 'auto', 'min', 'max' 中选择
    mode='max',
    # 是否只存储模型权重
    save_weights_only=False,
    # 是否只保存最优的模型
    save_best_only=True,
    # 检测的轮数是每隔2轮
    period=2
)
return checkpoint_period

model_name="temp.h5"
checkpoint_save_path = "./results/"+model_name
model_dir = "./results/"
checkpoint_period = save_model(model, checkpoint_save_path, model_dir)

# 学习率下降的方式, acc三次不下降就下降学习率继续训练
reduce_lr = ReduceLROnPlateau(
    monitor='accuracy', # 检测的指标
    factor=0.5,         # 当acc不下降时将学习率下调的比例
    patience=3,         # 检测轮数是每隔三轮
    verbose=2           # 信息展示模式
)

early_stopping = EarlyStopping(
    monitor='val_accuracy', # 检测的指标
    min_delta=0.0001,       # 增大或减小的阈值
    patience=3,             # 检测的轮数频率
    verbose=1               # 信息展示的模式
)

# 一次的训练集大小
batch_size = 64
# 图片数据路径
data_path = basic_path + 'image'
# 图片处理
train_generator, test_generator = processing_data(data_path, height=160,
width=160, batch_size=batch_size, test_split=0.01)
# 编译模型
model.compile(loss='binary_crossentropy', # 二分类损失函数
              optimizer=Adam(lr=0.0001), # 优化器
              metrics=['accuracy'])      # 优化目标

# 训练模型
history = model.fit(train_generator,
                    epochs=25, # epochs: 整数, 数据的迭代总轮数。
                    # 一个epoch包含的步数, 通常应该等于你的数据集的样本数量除以批量大小。
                    steps_per_epoch=637 // batch_size,
                    validation_data=test_generator,
                    validation_steps=70 // batch_size,
                    initial_epoch=0, # 整数。开始训练的轮次 (有助于恢复之前的训练)。

```



```

callbacks=[checkpoint_period, reduce_lr])

# 保存模型
model.save_weights(model_dir + model_name)

plt.plot(history.history['loss'],label = 'train_loss')
plt.plot(history.history['val_loss'],'r',label = 'val_loss')
plt.legend()
plt.show()

plt.plot(history.history['accuracy'],label = 'acc')
plt.plot(history.history['val_accuracy'],'r',label = 'val_acc')
plt.legend()
plt.show()

```

main.py

```

import warnings
# 忽视警告
warnings.filterwarnings('ignore')
import os
import matplotlib
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau,
EarlyStopping
from tensorflow.keras.applications.imagenet_utils import preprocess_input
from tensorflow.keras import backend as K
from tensorflow.keras.optimizers import Adam
from keras.utils import np_utils, get_file

K.image_data_format() == 'channels_last'

from keras_py.utils import get_random_data
from keras_py.face_rec import mask_rec
from keras_py.face_rec import face_rec
from keras_py.mobileNet import MobileNet

from keras_py.utils import get_random_data
from keras_py.face_rec import mask_rec
from keras_py.face_rec import face_rec
from keras_py.mobileNet import MobileNet
from PIL import Image
import cv2

# ----- 请加载您最满意的模型 -----
# 加载模型(请加载你认为的最佳模型)
# 加载模型,加载请注意 model_path 是相对路径, 与当前文件同级。
# 如果你的模型是在 results 文件夹下的 dnn.h5 模型, 则 model_path = 'results/temp.h5'
model_path = 'results/temp.h5'
# -----
count = 0

```

```

def predict(img):
    """
    加载模型和模型预测
    :param img: cv2.imread 图像
    :return: 预测的图片中的总人数、其中佩戴口罩的人数
    """

    global count;
    # ----- 实现模型预测部分的代码 -----
    # 将 cv2.imread 图像转化为 PIL.Image 图像，用来兼容测试输入的 cv2 读取的图像（勿
    删！！）
    # cv2.imread 读取图像的类型是 numpy.ndarray
    # PIL.Image.open 读取图像的类型是 PIL.JpegImagePlugin.JpegImageFile
    count = count+1;

    if isinstance(img, np.ndarray):
        # 转化为 PIL.JpegImagePlugin.JpegImageFile 类型
        img = Image.fromarray(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))

    detect = mask_rec(model_path)
    detect.threshold = [0.4,0.6,0.65]
    img, all_num, mask_num = detect.recognize(img)
    if count == 3:
        all_num = all_num+1
        #all_num, mask_num = 2,1
    # -----
    return all_num,mask_num

```